

Software Deployment

IE University - BCSAI - SDD - 2025

1. Introduction to Software Deployment

Software deployment is the process of **releasing software to users** in a controlled, repeatable, and reliable manner.

From code to production:

- Package code into deployable artifacts
- Provision and configure infrastructure
- Install and verify the application
- Route traffic and monitor health

Evolution: Manual, error-prone processes → Highly automated pipelines enabling multiple releases per day

Manual vs Automated Deployment

Aspect	Manual	Automated
Consistency	Varies by operator	Identical each time
Speed	Slow (hours)	Fast (minutes)
Errors	Typo-prone	Reproducible
Rollback	Stressful	Scripted
Auditability	Documentation gaps	Logged every step
Scale	Limited by humans	Unlimited

Automation = scripts + tools + pipelines that handle build, test, and deployment with minimal human input.

Goals of Modern Deployment

- **Reliability and consistency** across all environments
- **Minimize manual errors** through automation
- **Fast iteration** with small, frequent changes
- **Safe rollback** mechanisms
- **Transparency** with clear version tracking
- **Infrastructure parity** using code (IaC)

2. The Software Deployment Process

Typical stages:

1. **Source control** → Git as single source of truth
2. **Build** → compile, test, package in clean CI environment
3. **Test** → unit, integration, security checks
4. **Package** → create immutable artifacts (images, archives)
5. **Deploy** → provision infrastructure, install, configure
6. **Verify** → health checks, smoke tests
7. **Monitor** → metrics, logs, alerts

Environments: Dev → Staging → Production

Development:

- Fast feedback, instability expected
- Local or shared, simple configuration

Staging (Pre-production):

- Production-like infrastructure
- Realistic data and configuration
- Final validation before production

Production:

- Hardened, monitored, highly available
- Real users and data
- Strict access controls

Versioning and Artifacts

Semantic Versioning: MAJOR.MINOR.PATCH

- Git tags identify exact release commits
- Branching strategies: trunk-based, GitHub Flow, Git Flow

Artifacts:

- Immutable, versioned bundles (build once, deploy many)
- Container images (Docker)
- Archives (ZIP/JAR/WAR)
- Platform packages (npm, pip, NuGet)

Best practices:

- Tag with commit SHA and version
- Store in registries (ACR, Docker Hub, npm)

Deployment Pipeline Overview

```
Developer Commit → Git Push
↓
CI/CD Triggered (GitHub Actions)
↓
Build & Test (unit, integration, security)
↓
Package Artifact (Docker image, ZIP)
↓
Deploy to Staging → Verify
↓
Deploy to Production → Health Checks
↓
Monitor (metrics, logs, alerts)
↓
Rollback if Issues Detected
```


3. Continuous Integration / Continuous Deployment (CI/CD)

Continuous Integration (CI):

- Merge changes to main branch frequently
- Automated build + test on every push/PR
- Fast feedback (<10 min for core checks)
- Fail fast, fix immediately

Continuous Deployment (CD):

- Automatically promote green builds through environments
- Can be fully automated or require approval gates
- Enables multiple releases per day

Together: Fast feedback, small releases, reduced risk

CI/CD Pipeline Architecture

Building blocks:

- **Triggers:** push, pull_request, tag, schedule
- **Runners:** hosted or self-hosted agents
- **Jobs/Stages:** build → test → package → deploy → verify
- **Artifacts:** share build outputs between jobs
- **Environments:** dev → staging → prod with approvals
- **Secrets:** injected at runtime (never in code)

Benefits:

- Speed and consistency
- Higher quality through automation
- Better observability and auditability

4. GitHub Actions for Deployment Automation

GitHub Actions provides:

- Native integration with Git repositories
- YAML-based workflow definitions in `.github/workflows/`
- Hosted runners (Linux/Windows/macOS)
- Secrets management
- Rich marketplace of reusable actions
- Free tier for public repos

Why use it:

- No separate CI/CD service needed
- Built-in auth with GitHub tokens
- Tight integration with PRs, issues, releases

GitHub Actions Core Concepts

Term	Description
Workflow	Automated process defined in YAML
Job	Set of steps running on a runner (can run in parallel)
Step	Individual command or action
Runner	VM executing jobs (GitHub-hosted or self-hosted)
Action	Reusable unit from Marketplace or custom
Trigger	Event starting workflow (<code>push</code> , <code>pull_request</code> , etc.)

Workflow Triggers

Common triggers:

- `push` : on commits to specified branches
- `pull_request` : on PR open/update/merge
- `workflow_dispatch` : manual trigger via UI/API
- `release` : when a release is published
- `schedule` : cron-based (nightly builds)

Example: GitHub Actions Deployment Workflow

```
name: Deploy to Azure
on:
  push:
    branches: [main]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: '18'
          cache: 'npm'

      - run: npm ci
      - run: npm test
      - run: npm run build

      - name: Deploy to Azure Web App
        uses: azure/webapps-deploy@v2
        with:
          app-name: my-webapp
          publish-profile: ${ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }
          package: ./dist
```

GitHub Actions Marketplace

Popular actions:

- `actions/checkout@v4` - clone repository
- `actions/setup-node@v4` - install Node.js
- `actions/cache@v4` - cache dependencies
- `docker/build-push-action@v5` - build Docker images
- `azure/webapps-deploy@v2` - deploy to Azure
- `aws-actions/configure-aws-credentials@v4` - AWS auth

Custom actions:

- JavaScript/TypeScript actions
- Docker container actions
- Composite actions (combine steps)

Secrets and Environment Variables

Secrets: Encrypted values in repo/org settings

- Access via `${{ secrets.SECRET_NAME }}`
- Never log or expose in output
- Common: `AZURE_CREDENTIALS` , `AWS_ACCESS_KEY_ID` , API keys

Environment variables: Set at workflow/job/step level

```
env:  
  NODE_ENV: production  
  API_URL: https://api.example.com
```

Environments: Deployment targets with protection rules

- Required approvals
- Wait timers
- Branch restrictions

5. Deploying to Azure

Azure deployment targets:

Service	Type	Best For
Web Apps	PaaS	Traditional web apps, APIs
Container Apps	Serverless containers	Microservices, event-driven
AKS	Kubernetes	Complex orchestration
Functions	Serverless compute	Event-driven tasks
Static Web Apps	Static hosting	SPAs, JAMstack

Choose based on: control needed, scaling requirements, cost model

Azure Web Apps

Features:

- Fully managed PaaS
- Auto-scaling, deployment slots
- Built-in CI/CD integration
- Multiple runtime support

Deployment methods:

- Publish profiles (simpler)
- Service principals (recommended for production)
- GitHub Actions integration
- ZIP deploy, local Git

Azure Examples

Blue-Green with slots:

```
az webapp deployment slot swap \  
  --resource-group rg --name my-app --slot staging
```

Azure Container Apps

Serverless container hosting:

- Scale to zero (pay per use)
- Traffic splitting for canary releases
- Revision management
- Dapr integration

Azure Examples

Example deployment:

```
az containerapp update \  
  --name my-app --resource-group rg \  
  --image myregistry.azurecr.io/myapp:latest
```

Azure Examples

Canary release:

```
az containerapp ingress traffic set \  
  --name my-app --resource-group rg \  
  --revision-weight latest=10 previous=90
```

Authentication: Publish Profiles vs Service Principals

Publish Profiles:

- XML file with credentials
- Download from Azure Portal
- Scoped to single resource
- Easier for beginners

Service Principals (recommended):

- Azure AD identity with RBAC
- Can access multiple resources
- Easier to audit and rotate
- Supports OIDC (keyless auth)

Best practice: Use service principals with OIDC for production

6. Deployment Strategies

Strategy	Downtime	Cost	Complexity	Rollback	Best For
Recreate	Yes	Low	Low	Slow	Dev/test
Rolling	No	Low	Medium	Medium	Standard prod
Blue-Green	No	High	Medium	Instant	Mission-critical
Canary	No	Medium	High	Fast	High-traffic, risk-averse

Choosing:

- Standard web app → Rolling update
- Can't afford downtime → Blue-green
- Large user base → Canary

Recreate Deployment

Process:

1. Stop all instances
2. Deploy new version
3. Start instances

Pros: Simple, clean state

Cons: Downtime, all-or-nothing

When to use: Dev environments, maintenance windows, internal tools

```
docker-compose down  
docker-compose pull  
docker-compose up -d
```

Rolling Update

Process:

1. Deploy to subset of instances (25%)
2. Health check
3. Route traffic
4. Repeat for remaining instances

Pros: Zero downtime, gradual rollout

Cons: Mixed versions, slower

Kubernetes example:

```
strategy:  
  type: RollingUpdate  
  rollingUpdate:  
    maxUnavailable: 1  
    maxSurge: 1
```

Blue-Green Deployment

Process:

1. Blue (current) serves production
2. Deploy to Green (new)
3. Test Green thoroughly
4. Switch traffic Blue → Green
5. Keep Blue for instant rollback

Pros: Zero downtime, instant rollback, full testing

Cons: 2x infrastructure cost

Azure example:

```
az webapp deployment slot swap \  
  --resource-group rg --name my-app --slot staging
```

Canary Release

Process:

1. Deploy new version alongside old
2. Route 5% traffic to canary
3. Monitor metrics (errors, latency)
4. Gradually increase: 10% → 25% → 50% → 100%
5. Rollback if issues detected

Pros: Minimal blast radius, data-driven

Cons: Requires sophisticated routing and monitoring

Canary Release Traffic Progression Example

```
Hour 0: 5% canary    → monitor  
Hour 1: 10% canary   → if metrics OK  
Hour 4: 50% canary  
Hour 8: 100% canary → full rollout
```

Feature Flags and Dark Launches

Feature flags:

- Enable/disable features at runtime
- Deploy code without exposing functionality
- A/B testing, gradual rollouts
- Emergency kill switches

Dark launch:

- Deploy new code in production
- Enable only for internal users or test traffic
- Validate performance before release

Feature Flags and Dark Launches

Tools: LaunchDarkly, Unleash, ConfigCat

Best practices:

- Keep flags short-lived
- Clean up old flags
- Don't nest deeply

7. Infrastructure as Code (IaC)

Concept: Manage infrastructure through **declarative code**, not manual clicks

Traditional approach:

1. Log into cloud portal
2. Click through UI
3. Configure manually
4. Document (often forgotten)

IaC approach:

1. Write infrastructure definition
2. Commit to version control
3. Run automated tool
4. Same code for dev/staging/prod

Benefits of IaC

Key advantages:

- **Repeatability:** Same code = identical infrastructure
- **Version control:** Track changes in git, review via PRs
- **Speed:** Provision environments in minutes
- **Documentation as code:** Always up-to-date
- **Disaster recovery:** Recreate from code
- **Cost management:** Spin down non-prod environments

Trade-offs:

- Learning curve
- Initial setup investment
- Discipline to avoid manual changes

IaC Tools Comparison

Tool	Multi-cloud	Language	Best For
Terraform	YES	HCL	Multi-cloud, industry standard
Bicep	NO (Azure)	Bicep	Azure-specific projects
Pulumi	YES	TypeScript/Python/Go	Devs wanting familiar languages
ARM/CloudFormation	NO	JSON/YAML	Cloud-native, simple setups

Terraform Workflow

Core commands:

1. `terraform init` - Download providers
2. `terraform plan` - Preview changes
3. `terraform apply` - Create/update resources
4. `terraform destroy` - Tear down

Key concepts:

- **Providers:** Plugins (azurerm, aws, google)
- **Resources:** Infrastructure components
- **State:** Record of deployed resources
- **Variables:** Parameterize configs
- **Outputs:** Export values

Example: Terraform for Azure Web App

```
terraform {
  required_providers {
    azurerm = { source = "hashicorp/azurerm", version = "~> 3.0" }
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "rg" {
  name     = "rg-myapp-prod"
  location = "East US"
}

resource "azurerm_service_plan" "plan" {
  name                = "asp-myapp-prod"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  os_type             = "Linux"
  sku_name            = "B1"
}

resource "azurerm_linux_web_app" "app" {
  name                = "app-myapp-prod"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  service_plan_id     = azurerm_service_plan.plan.id

  site_config {
    application_stack {
      node_version = "18-lts"
    }
  }
}
```

Bicep: Azure's Modern IaC

Advantages:

- Simpler syntax than ARM JSON
- Excellent VS Code extension
- Day-0 support for new Azure features
- Compiles to ARM templates

Bicep Example

```
param appName string = 'myapp'
param environment string = 'prod'

resource appServicePlan 'Microsoft.Web/serverfarms@2022-03-01' = {
  name: 'asp-${appName}-${environment}'
  location: 'eastus'
  sku: { name: 'B1' }
  kind: 'linux'
}

resource webApp 'Microsoft.Web/sites@2022-03-01' = {
  name: 'app-${appName}-${environment}'
  location: 'eastus'
  properties: {
    serverFarmId: appServicePlan.id
    siteConfig: {
      linuxFxVersion: 'NODE|18-lts'
    }
  }
}
```

Integrating IaC with CI/CD

Workflow:

1. Code change triggers pipeline
2. Provision/update infrastructure (Terraform/Bicep)
3. Deploy application to infrastructure
4. Run automated tests
5. Promote to next environment

Example: GitHub Actions + Terraform

```
jobs:
  infrastructure:
    steps:
      - uses: hashicorp/setup-terraform@v3
      - run: terraform init
      - run: terraform plan -out=tfplan
      - run: terraform apply -auto-approve tfplan
      - id: outputs
        run: echo "app_name=$(terraform output -raw app_name)" >> $GITHUB_OUTPUT
    outputs:
      app_name: ${ steps.outputs.outputs.app_name }

  deploy:
    needs: infrastructure
    steps:
      - uses: azure/webapps-deploy@v2
        with:
          app-name: ${ needs.infrastructure.outputs.app_name }
```


Best Practices for IaC in CI/CD

Recommended approach:

- Separate jobs: infrastructure first, then app deployment
- Use remote state (Azure Storage, S3, Terraform Cloud)
- Store cloud credentials in GitHub Secrets
- Run `terraform plan` on pull requests for review
- Apply only on merge to main branch
- Environment protection rules for production
- Scheduled drift detection jobs

Best Practices for IaC in CI/CD

Pull request workflow:

- PR opened → `terraform plan` runs, posts plan as comment
- Team reviews infrastructure changes
- PR merged → `terraform apply` runs automatically
- Infrastructure versioned alongside application code

8. Monitoring and Rollback

After deployment:

- Monitor performance and logs (Azure Monitor, Application Insights)
- Set up automated alerts on failure or degradation
- Track deployment frequency and success rate

Rollback options:

1. Redeploy previous version (artifact/tag)
2. Switch to previous environment (blue-green)
3. Revert git commit and trigger pipeline
4. Use feature flags to disable problematic features

Best practices:

- Keep rollback procedures documented and rehearsed

9. Summary

Software Deployment Process:

- Source → Build → Test → Package → Deploy → Verify → Monitor

Automation with CI/CD:

- GitHub Actions provides native integration
- Automated pipelines reduce errors and speed delivery

Deployment Strategies:

- Recreate (simple), Rolling (gradual), Blue-Green (instant rollback), Canary (data-driven)
- Choose based on downtime tolerance and risk

Infrastructure as Code:

- Terraform (multi-cloud) and Bicep (Azure)

10. Key Takeaways

1. **Automation is essential** - reduces human error, accelerates releases
2. **Small, frequent deployments** are safer than big releases
3. **GitHub Actions integrates CI/CD** directly with repositories
4. **Multiple deployment strategies** exist - choose the right one for your needs
5. **IaC ensures reproducibility** and enables disaster recovery
6. **Monitor and have rollback plans** - deployments don't end at "deploy"

The deployment automation cycle:

Code → CI → Infrastructure provisioning → Application deployment →
Production → Monitor